מחלקה להנדסת תוכנה

BRAUDE College of Engineering, karmiel

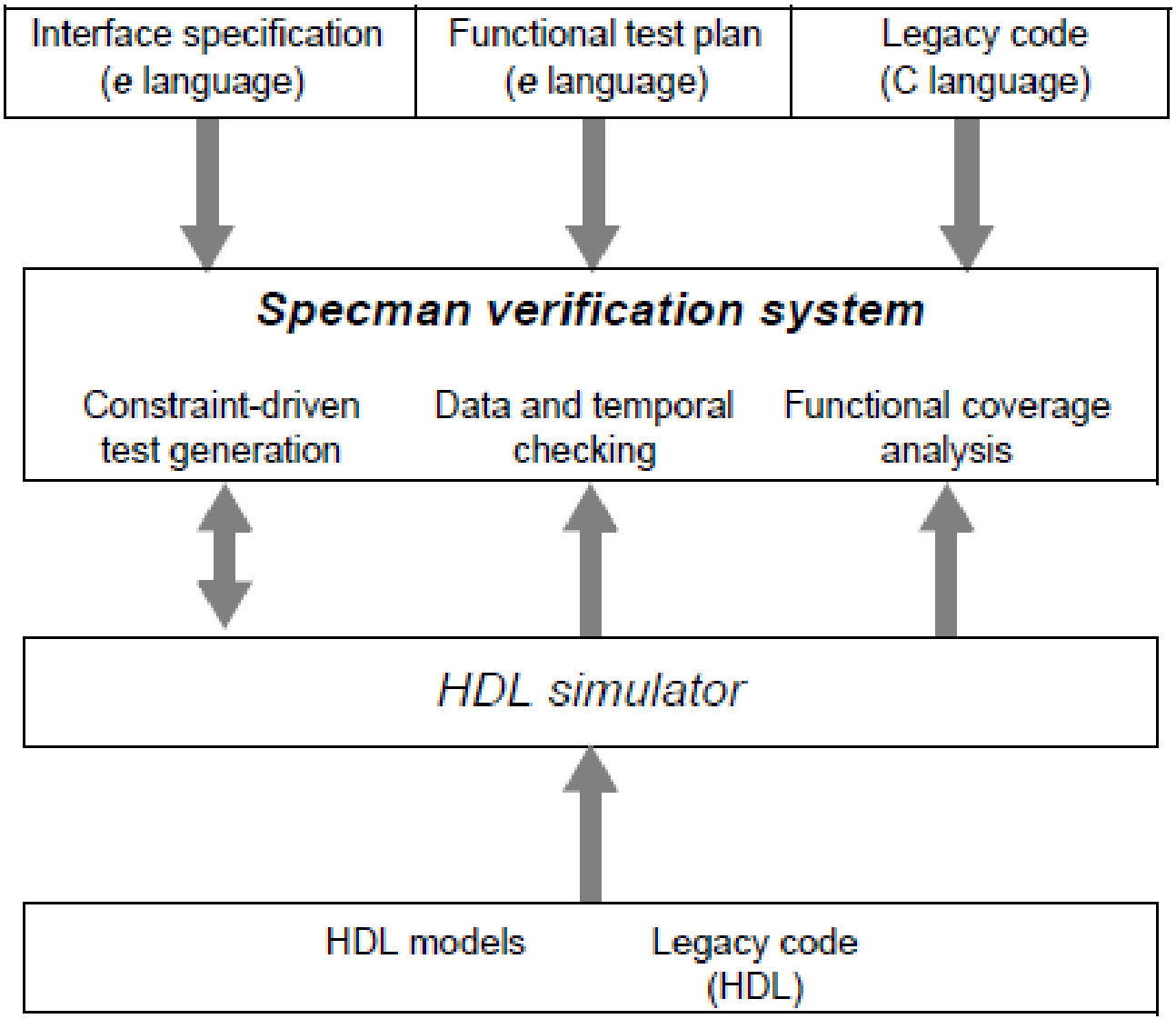
בדיקת מערכות ספרתיות ואימות תכנון

פרק 3

cadence™

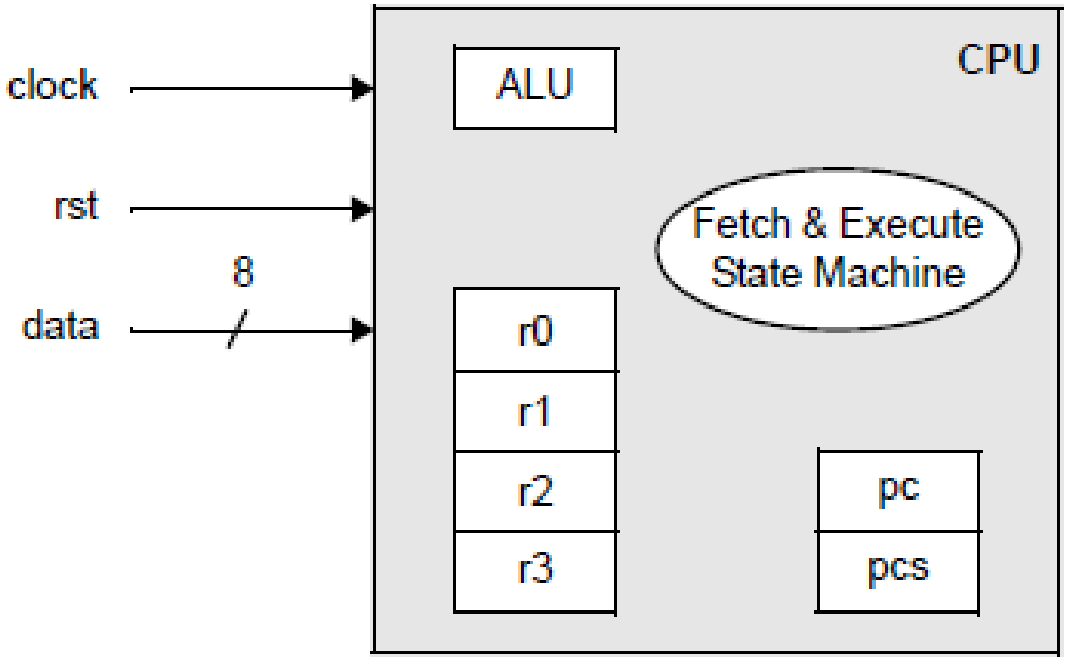
SPECMAN

The Specman System Verification



The Design Specifications

- The device under test (DUT) is an 8-bit CPU with a reduced instruction set



Available Instructions

Arithmetic:
ADD, ADDI, SUB, SUBI

Logic:
AND, ANDI, XOR, XORI

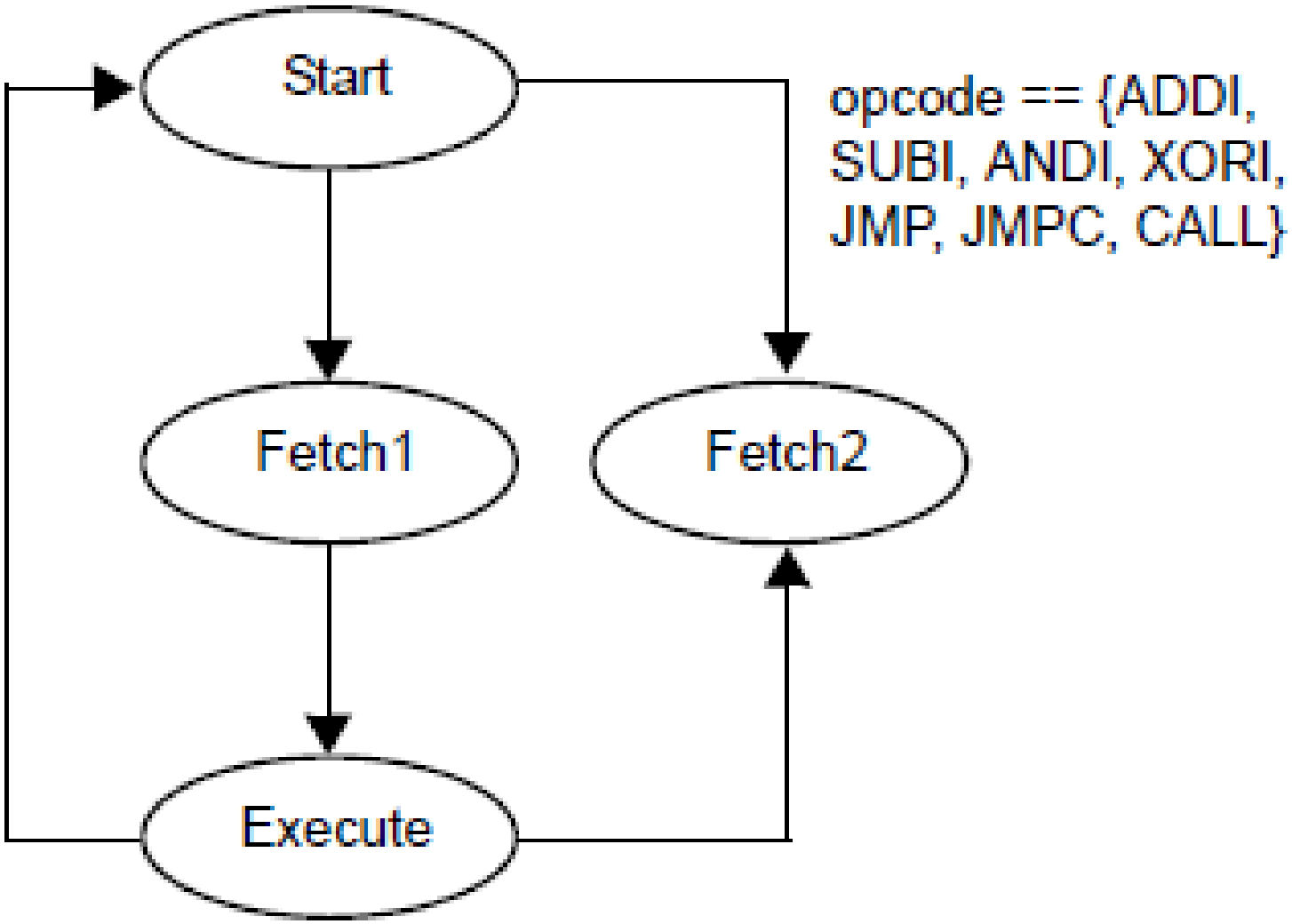
Control Flow:
JMP, JMPC, CALL, RET

No-Operation:
NOP

The Design Specifications

- The state machine diagram for the CPU.
- The second fetch cycle is only for *immediate* instructions and for instructions that control execution flow.
- There is a 1-bit signal associated with each state, *exec*, *fetch2*, *fetch1*, *strt*.
- If no reset occurs, the *fetch1* signal must be asserted exactly one cycle after entering the execute state.

The Design Specifications



The Interface Specifications

- All instructions have a 4-bit opcode and two operands.
- The first operand specifies one of four 4-bit registers internal to the CPU.
- The second operand depends on the type of instruction:
  - **Register instructions—**The second operand specifies another one of the four internal registers.
  - **Immediate instructions—**The second operand is an 8-bit value. When the opcode is of type JMP, JMPC, or CALL, this operand must be a 4-bit memory location.

Register Instruction

byte	1							
bit	7	6	5	4	3	2	1	0
	opcode				op1		op2	

Immediate Instruction

byte	1								2							
bit	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
	opcode				op1		don't care		op2							

Test 1

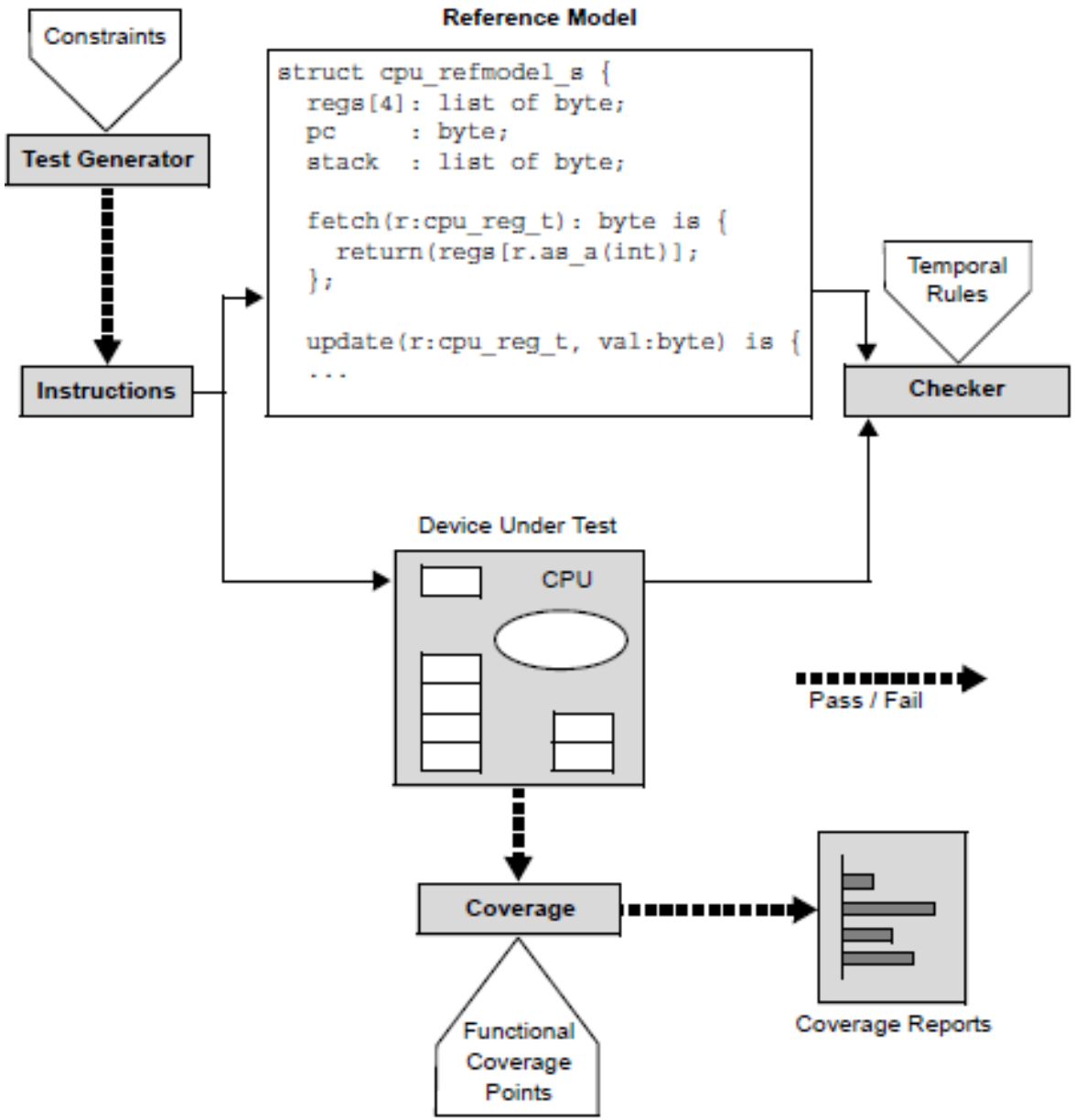
A simple go/no-go confirming that the verification environment is working properly.

- Generate five instructions.
- Use either the ADD or ADDI opcode.
- Set op1 to REG0.
- Set op2 either to REG1 for a register instruction or to value 0x5 for an immediate instruction.

Overview of the Verification Environment

- Constrain the Specman test generator to creation of valid CPU instructions.
- Compare the program counters in the CPU to those in a reference model.
- Define temporal rules to check the DUT behavior.
- Define coverage points for state machine transitions and instructions.

Design Verification Environment Block-Level Diagram



Creating the CPU Instruction Structure

Construct	How the Construct is Used
<'...'>	Marks the beginning and end of *e* code.
struct	Creates a data structure to hold the CPU instructions.
extend	Adds the data structure containing the CPU instructions to the Specman system of data structures.

Construct	How the Construct is Used
list of	Creates an array or list without having to keep track of pointers or allocate memory.
type	Defines an enumerated data type for the CPU instructions.
bits	Defines the width of an enumerated type.
keep	Specifies rules or constraints for the instruction fields.
when	Implements conditional constraints on the possible values of the instruction fields.

Creating the CPU Instruction Structure

defines the legal opcodes as an enumerated type

defines the internal registers

when complete, this structure defines a valid CPU instruction

// indicates that rest of line is a comment

when complete, this construct adds the CPU instruction set to the Specman *system*

```
<'
type cpu_opcode_t: [ // Opcodes
    ADD, ADDI, SUB, SUBI,
    AND, ANDI, XOR, XORI,
    JMP, JMPC, CALL, RET,
    NOP
] (bits: 4);

type cpu_reg_t: [ // Register names
    REG0, REG1, REG2, REG3
] (bits:2);

struct cpu_instr_s {

    // defines 2nd op of reg instruction

    // defines 2nd op of imm instruction

    // defines legal opcodes for reg instr

    // defines legal opcodes for imm instr

    // ensures 4-bit addressing scheme

};

extend sys {
    // creates the stream of instructions

};
'>
```

Define two fields in the *cpu_instr_s* structure

add these two lines into the file

```
struct cpu_instr_s {
    %opcode     :cpu_opcode_t;
    %op1        :cpu_reg_t;

    // defines 2nd op of reg instruction
.
.
.
};
```

Define a field for the second operand

add this line to define the field 'kind' and define an enumerated type at the same time

```
struct cpu_instr_s {
    %opcode     :cpu_opcode_t;
    %op1        :cpu_reg_t;
    kind        :[imm, reg];

    // defines 2nd op of reg instruction
.
.
.
};
```

Define the conditions

Define the conditions under which the second operand is a register and those under which it is a byte of data.

```
struct cpu_instr_s {
    %opcode     :cpu_opcode_t;
    %op1        :cpu_reg_t;
    kind        :[imm, reg];

    // defines 2nd op of reg instruction
    when reg cpu_instr_s {
        %op2    :cpu_reg_t;
    };

    // defines 2nd op of imm instruction
    when imm cpu_instr_s {
        %op2    :byte;
    };
.
.
.
};
```

Constrain on the opcodes

Whenever the opcode is one of the register opcodes, then the *kind* field must be *reg*. Whenever the opcode is one of the immediate opcodes, then the *kind* field must be *imm*. You can use the **keep** construct with the implication operator => to easily create these complex constraints.

```
struct cpu_instr_s {
.
.
.
    // defines legal opcodes for reg instr
    keep opcode in [ADD, SUB, AND, XOR, RET, NOP]
        => kind == reg;

    // defines legal opcodes for imm instr
    keep opcode in [ADDI, SUBI, ANDI, XORI, JMP, JMPC, CALL]
        => kind == imm;

    // ensures 4-bit addressing scheme

};
```

Constrain on the second operand

Constrain the second operand to a valid memory location (less than 16) when the instruction is immediate.

```
struct cpu_instr_s {
.
.
.
    // ensures 4-bit addressing scheme
    when imm cpu_instr_s {
        keep opcode in [JMP, JMPC, CALL] => op2 < 16;
    };
};
```

Creating the List of Instructions

```
extend sys {
    // creates a stream of instructions
    !instrs: list of cpu_instr_s;
};
```

- The exclamation point preceding the field name *instrs* tells the Specman system to create an empty data structure to hold the instructions.
- Then, each test tells the system when to generate values for the list, either before simulation (pre-run generation) or during simulation (on-the-fly generation).

Generating the First Test

A simple go/no-go confirming that the verification environment is working properly.

- Generate five instructions.
- Use either the ADD or ADDI opcode.
- Set op1 to REG0.
- Set op2 either to REG1 for a register instruction or to value 0x5 for an immediate instruction

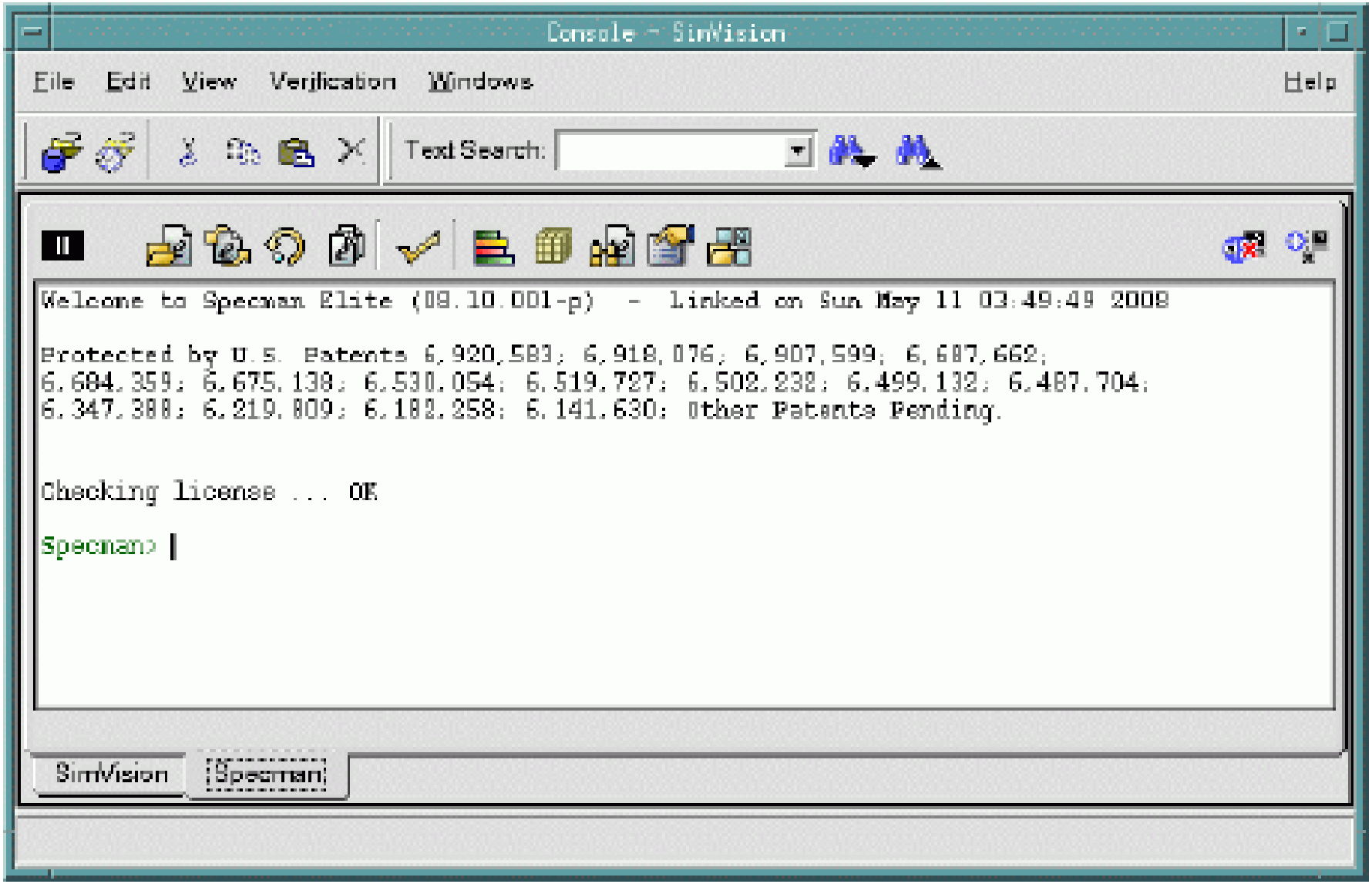
Defining the Test Constraints

constrains the opcode and operands

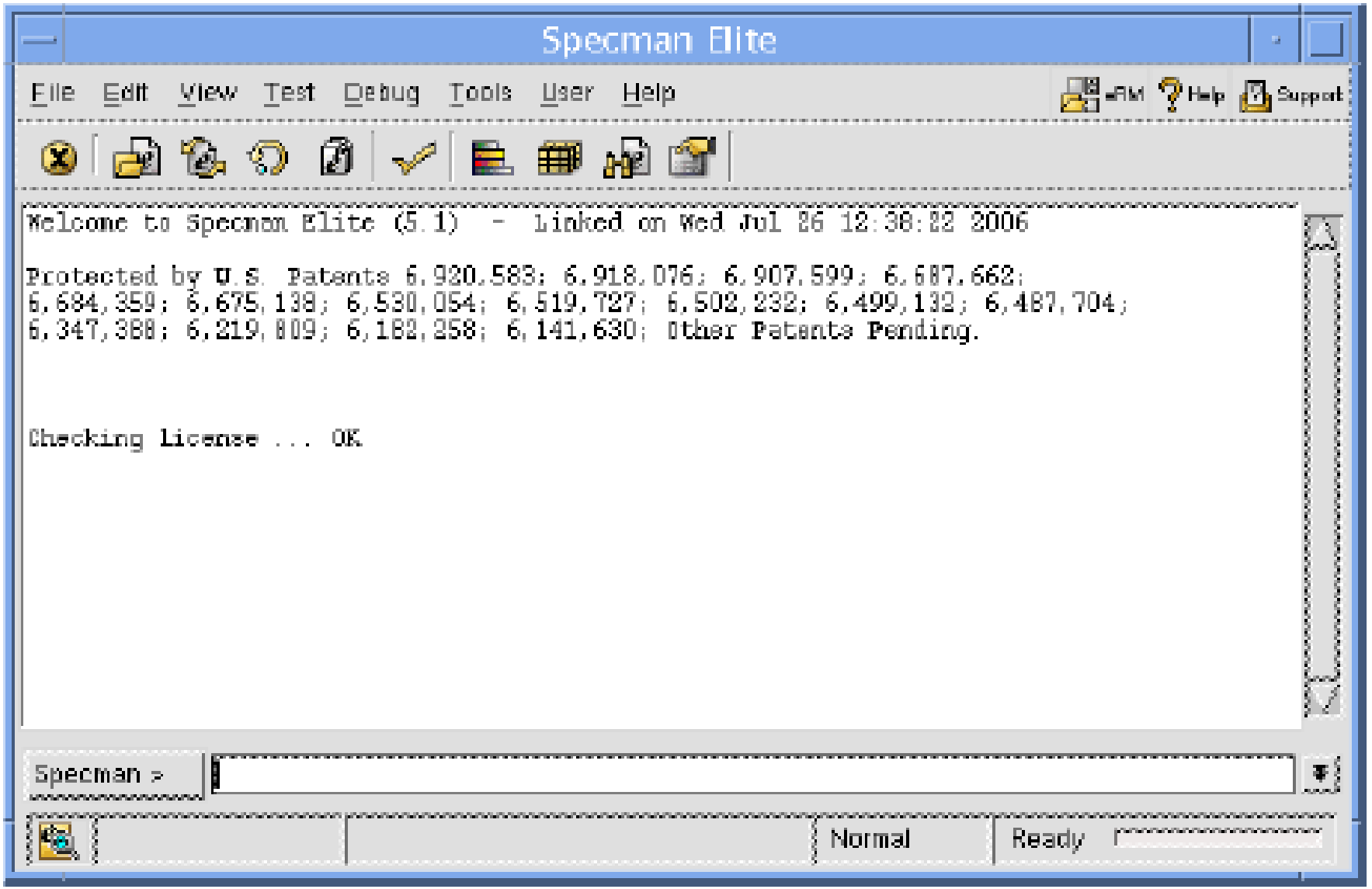
constrains the number of instructions

```
<'
extend cpu_instr_s {
    //test constraints
    keep opcode in [ADD, ADDI];
    keep op1 == REG0;
    when reg cpu_instr_s { keep op2 == REG1; };
    when imm cpu_instr_s { keep op2 == 0x5; };
};

extend sys {
    //generate 5 instructions
    keep instrs.size() == 5;
};
```

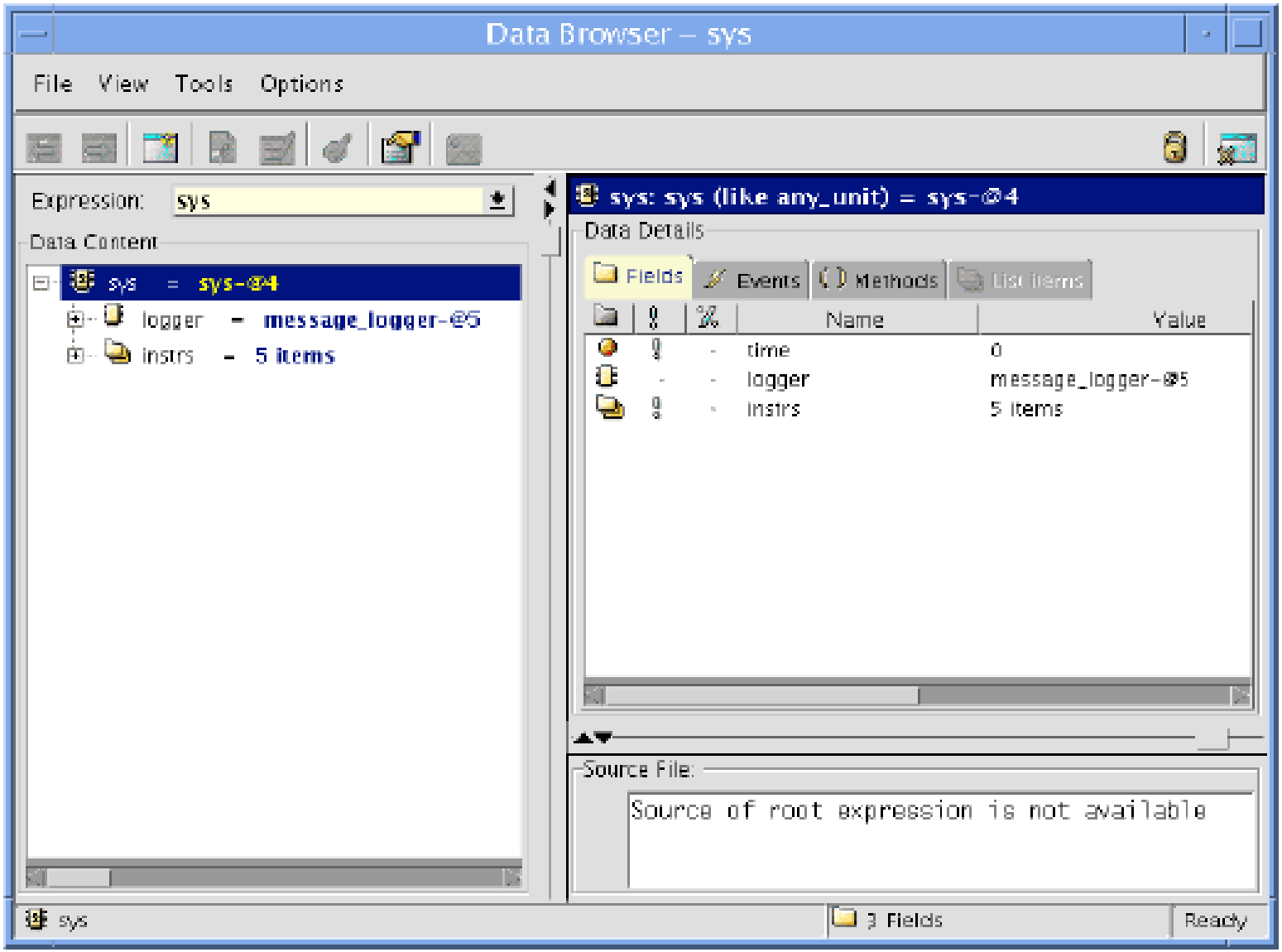
SimVison's Specman Tab



Specview Main Window

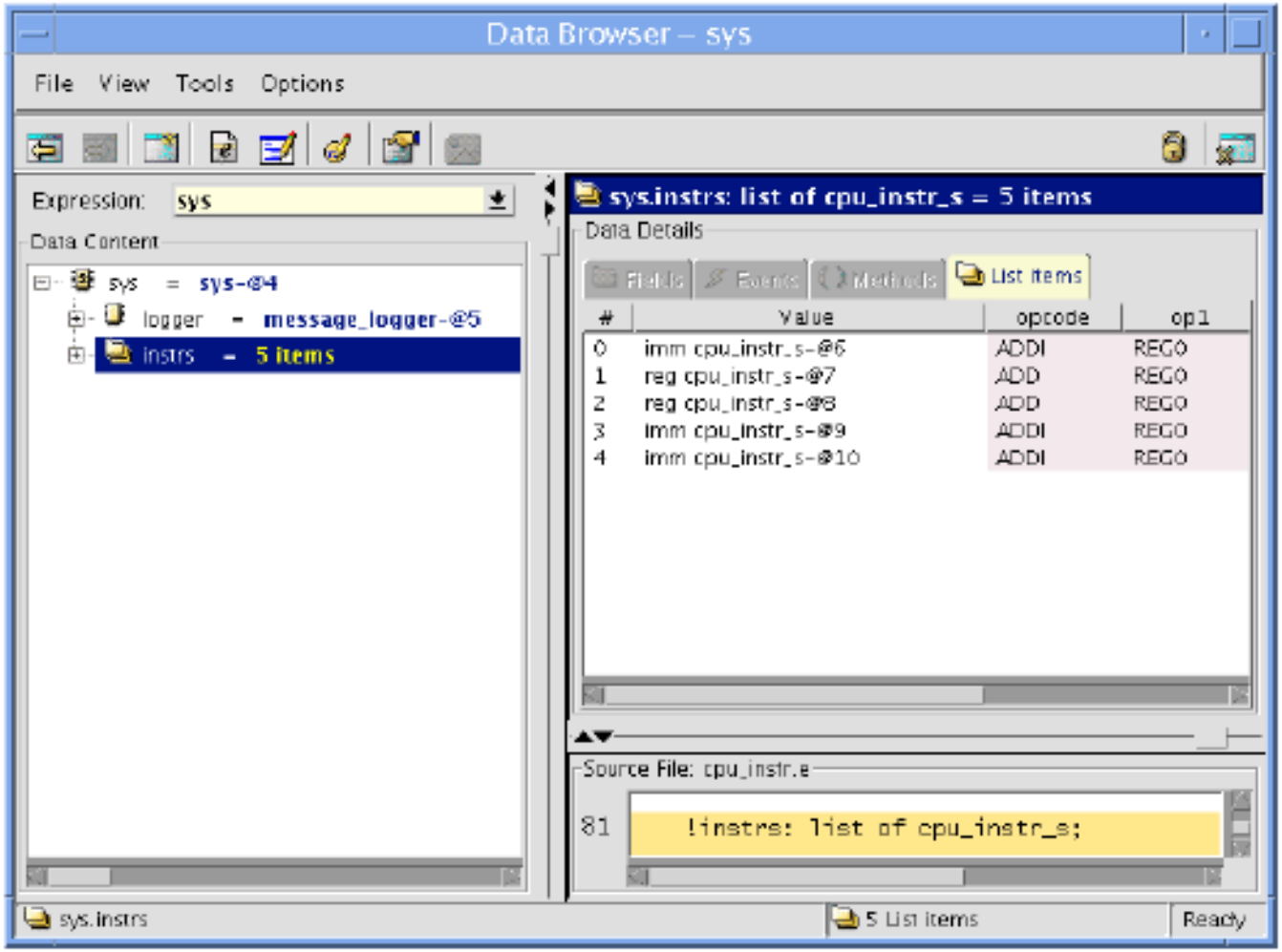


Generating the Test



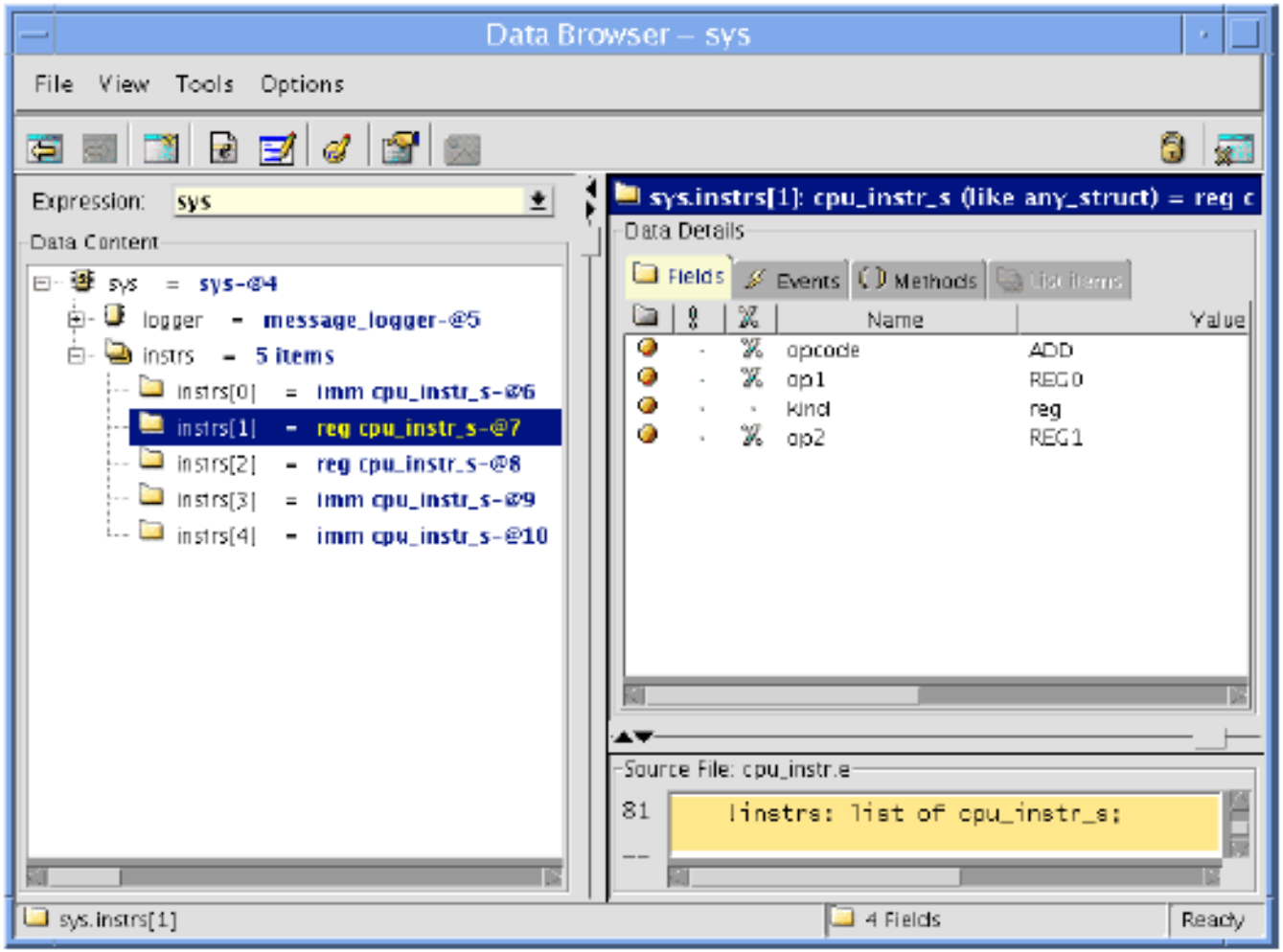
The list of five generated instructions

The list of five generated instructions appears in the top right pane.

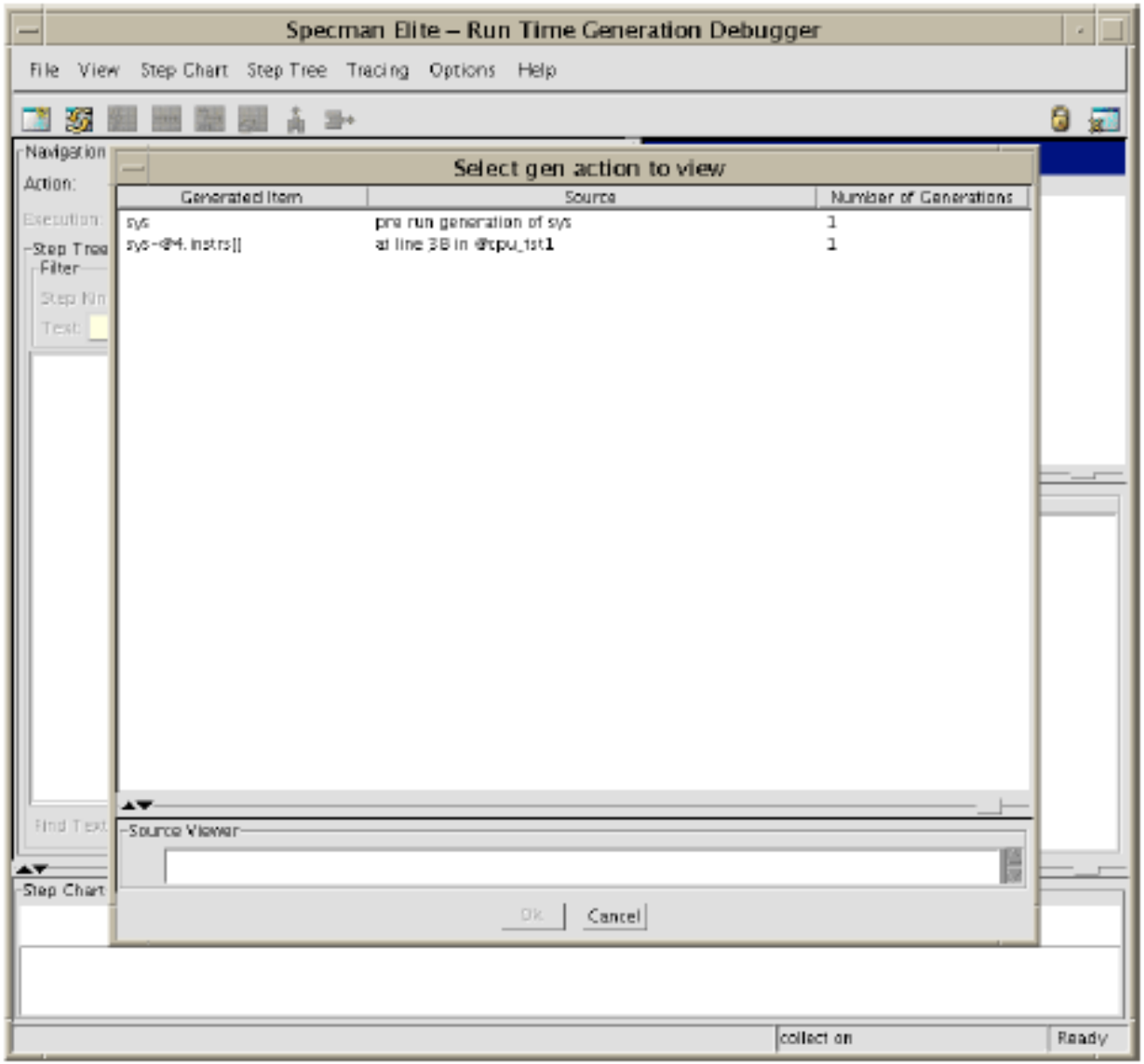


The generated values for the first *reg*

The generated values for the fields of the first *reg* instruction object appear in the right pane.



Analyzing Generation



Driving and Sampling the DUT

- **Time consuming methods (TCMs)—**You can write procedures in ***e*** that are synchronized to other TCMs or to an HDL clock. You can use these procedures to drive and sample test data.

- **DUT signal access—**You can easily access signals and variables in the DUT, either for driving and sampling test data or for synchronizing TCMs.

Driving and Sampling the DUT

- • **Simulator interface automation—**You can drive and sample a DUT without having to write PLI (Verilog simulators) or FLI/CLI (VHDL simulators) code. The Specman system automatically creates the necessary PLI/FLI calls for you.

Driving and Sampling the DUT

emit	Triggers a named event from within a TCM.
@	Synchronizes the TCMs with an event.
event	Creates a temporal object, in this case a clock, that is used to synchronize the TCMs.
'*hdl_signal_name*'	Accesses a signal in the DUT.
method () **is…**	Creates a procedure (method) that is a member of a struct and manipulates the fields of that struct. Methods can execute in a single point of time, or they can be time consuming methods (TCMs).
pack ()	Converts data from higher level *e* structs and fields into the bit or byte representation expected by the DUT.
wait	Suspends action in a TCM until the expression is true.

Defining the Protocols

There are two protocols to define for the CPU:

1. **Reset protocol—**drives the *rst* signal into the DUT.
2. **Drive instructions protocol—**drives instructions into the DUT according to the correct protocol indicated by the *fetch1* and *fetch2* signals.

Defining the Protocols

- The drive instructions protocol has one TCM for *pre-run generation*, where the complete list of instructions is generated and then simulation starts.
- There is another TCM for *on-the-fly generation*, where signals in the DUT are sampled before the instruction is generated. The test in this chapter uses the simple methodology of pre-run generation, while subsequent tests in this tutorial use the more powerful on-the-fly generation.

TCMs Required to Drive the CPU

Name	Function
drive_cpu()	Calls *reset_cpu ()*. Then, depending on whether the list of CPU instructions is empty or not, it calls *gen_and_drive_instrs ()* or *drive_pregen_instrs ()*.
reset_cpu()	Drives the *rst* signal in the DUT to low for one cycle, to high for five cycles, and then to low.
gen_and_drive_instrs()	Generates the next instruction, and then calls *drive_one_instr ()*.
drive_pregen_instrs()	Calls *drive_one_instr ()* for each generated instruction.
drive_one_instr()	Sends the instruction to the DUT. If the instruction is an immediate instruction, it also waits for the *fetch2* signal to rise, and then sends the second byte of data. Last, it waits for the *exec* signal to rise.

Sample the DUT

The CPU architecture requires that tests drive and sample the DUT on the falling edge of the clock. Therefore, all TCMs are synchronized to *cpuclk*, which is defined as follows:

```
extend sys {
        event cpuclk is (fall('top.clk')@sys.any);
};
```

The drive_one_instr () TCM

```
drive_one_instr(instr: cpu_instr_s) @sys.cpuclk is {
        var fill0 : uint(bits : 2) = 0b00;

        wait until rise('top.fetch1');

        emit instr.start_drv_DUT;

        if instr.kind == reg then {
            'top.data' = pack(packing.high, instr);
        } else {
        // immediate instruction
            'top.data' = pack(packing.high, instr.opcode,
                instr.op1, fill0);
            wait until rise('top.fetch2');
            'top.data' = pack(packing.high, instr.imm'op2);
        };

        wait until rise('top.exec');

        // execute instr in refmodel
        // sys.cpu_refmodel.execute(instr, sys.cpu_dut);
};
```

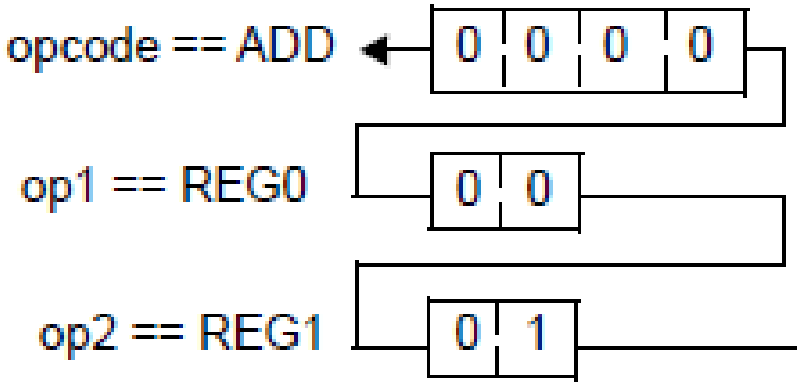
The drive_one_instr () TCM

- The *start_drv_DUT* event emitted by *drive_one_instr* is not used by any of the TCMs that drive the CPU. We will use it later chapter to trigger functional coverage analysis.

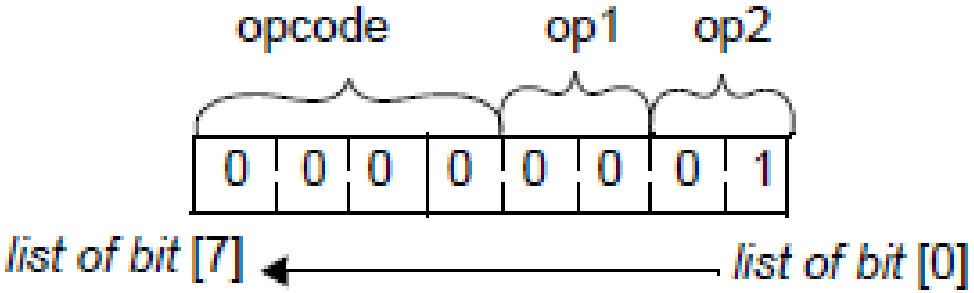
- The **pack()** function is a Specman built-in function that facilitates the conversion from higher level data structure to the bit stream required by the DUT.

A Register Instruction as Received by the DUT

The instruction struct with three fields:



The instruction packed into a bit stream, using the packing.high ordering



Generating Constraint-Driven Test

- **Directed-random test generation—**This feature lets you apply constraints to focus random test generation on areas of the design that need to be exercised the most.

- **Random seed generation—**Changing the seed used for random generation enables the Specman system to quickly generate a whole new set of tests.

Generating Constraint-Driven Test

Construct	How the Construct is Used
keep soft	Specifies a soft constraint that is kept only if it does not conflict with other hard **keep** constraints.
select	Used with **keep soft** to control the distribution of the generated values.
SimVision & (Specview) Command	**How the Command is Used**
Verification ›› Specman Configuration (Tools ›› Config)	Used to access the Generation tab of the Specman Configuration Options window for creating a user-defined seed for random test generation.
Verification ›› Save Specman State (File ›› Save)	Saves the current test environment, including the random seed, to a .esv file. You can load this file with the **Verification ›› Restore Specman State** (in Specview, **File ›› Restore**) command.
Verification ›› Test with Random Seed (Test ›› Test with Random Seed)	Generates a set of tests with a new random seed.

The steps for generating random tests

The steps for generating random tests are:

1. Defining weights for random tests.

2. Generating and running tests with a user-specified seed.

3. Generating and running tests with a random seed.

Defining Weights for Random Tests

Because of the way that CPUs are typically used, arithmetic and logical operations comprise a high percentage of the CPU instructions.

You can use the **select** construct with **keep soft** to require the Specman system to generate a higher percentage of instructions for arithmetic and logical operations than for control flow.

Test 2

Multiple random variations on a test gains high percentage coverage on commonly executed instructions.

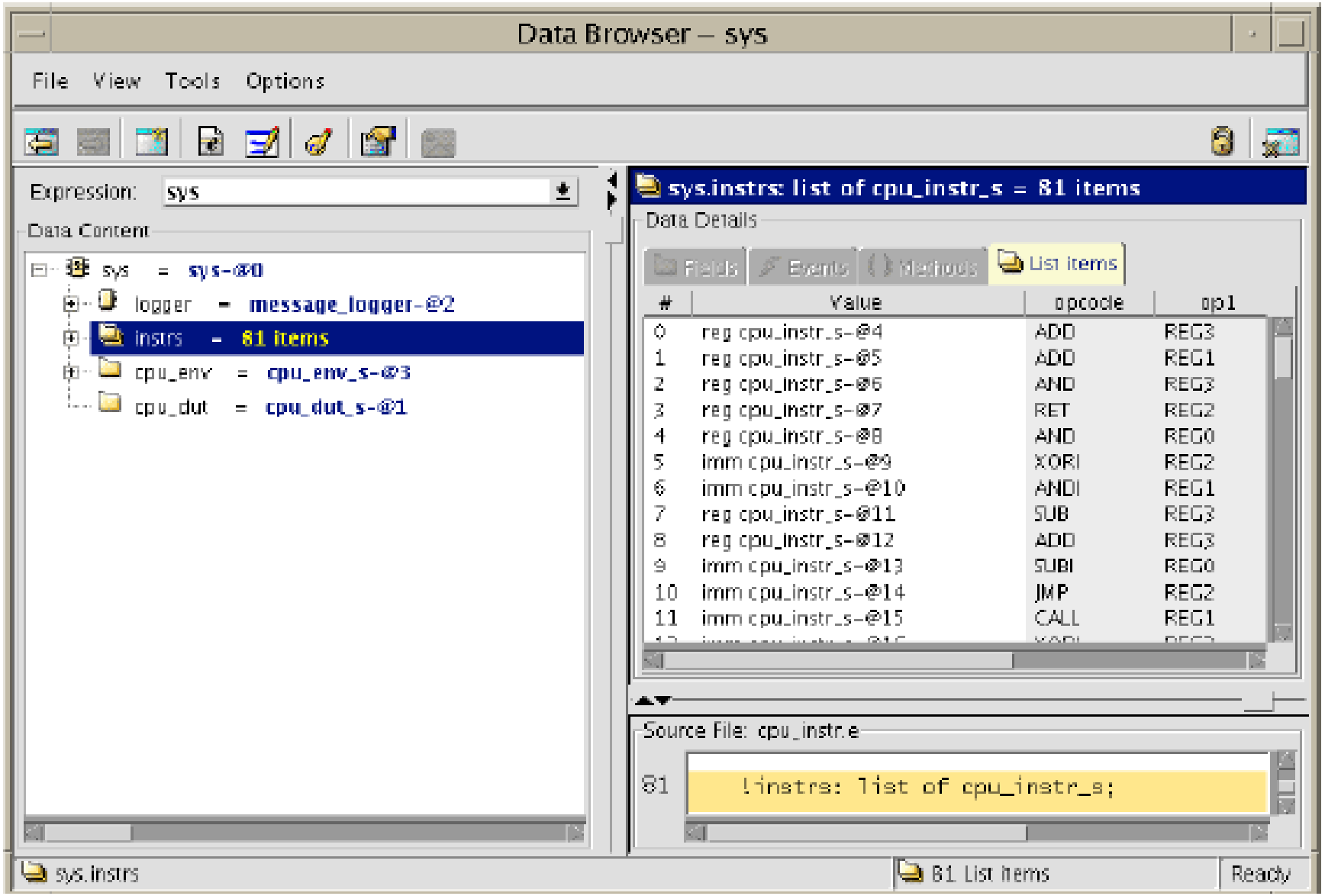
- Use constraints to direct random testing towards the more common arithmetic and logic operations rather than the control flow operations.
- Run the test many times, each time with a different random seed.

Weighted Constraints

puts equal weight on arithmetic and logical operations and less weight on control flow operations

```
<'
import cpu_top;

extend cpu_instr_s {
    keep soft opcode == select {
        10 : [JMP, JMPC, CALL, RET, NOP];
        30 : [ADD, ADDI, SUB, SUBI];
        30 : [AND, ANDI, XOR, XORI];
    };
};

'>
```

Generating Tests With a User-Specified Seed



Generating Tests With a Random Seed

To run a test using a Specman-generated random seed:

1. In SimVision, click **File ›› Reload *e* Files** (in Specview, **File ›› Reload)**.

2. Click Verification ›› **Test with Random Seed** (in Specview, **Test ›› Test with Random Seed)**.

The Specman system runs the test with the random seed shown in the Specman console window and reports the results.

Defining Coverage

Construct	How the Construct is Used
event	Defines a condition that triggers sampling of coverage data.
cover	Defines a group of data collection items.
item	Identifies an object to be sampled.

The three types of coverage data that you might want to collect are:

1. Coverage data for the finite state machine (FSM).
2. Coverage data for the generated instructions.
3. Coverage data for the corner case.

Defining Coverage for the FSM

defines the coverage group cpu_fsm

```
extend cpu_env_s {
    event cpu_fsm is @sys.cpuclk;

    // DUT Coverage: State Machine and State
    // Machine transition coverage
    cover cpu_fsm is {
      item fsm: cpu_FSM_type_t = 'top.cpu.curr_FSM';
      transition fsm;
    };
};
```

Construct	How the Construct is Used
transition	Identifies an object whose current and previous values are to be collected when the sampling event occurs.

Defining Coverage for the Generated Instructions

This coverage group uses a sampling event that is declared and triggered in the *cpu_drive.e* file.

```
drive_one_instr(instr: cpu_instr_s) @sys.cpuclk is {
...
        emit instr.start_drv_DUT;
...
```

Thus data collection for the instruction stream occurs each time an instruction is driven into the DUT.

Defining Coverage for the Generated Instructions

```
extend cpu_instr_s {

    cover start_drv_DUT is {
        item opcode;
        item op1;
    };
};
```

Test 3

Generation of a corner case test scenario that exercises JMPC opcode when the carry bit is asserted. Note that it is difficult to efficiently cover this scenario by purely random or purely directed tests.

- Generate many arithmetic opcodes to increase the chances of carry bit assertion.
- Monitor the DUT and use on-the-fly generation to generate many JMPC opcodes when the carry signal is high.

Defining Coverage for the Corner Case

Test 3 of the functional test plan specifies the corner case that you want to cover. To test the behavior of the DUT when the JMPC (jump on carry) instruction opcode is issued, you need to be sure that the JMPC opcode is issued only when the carry signal is high. Here, you define a coverage group so you can determine how often that combination of conditions occurs

Define coverage data for the corner case

```
extend cpu_instr_s {

    cover start_drv_DUT is {
        item opcode;
        item op1;
        item carry: bit = 'top.carry';
    };
};
```

Define a cross item between opcode and carry

```
extend cpu_instr_s {

    cover start_drv_DUT is {
        item opcode;
        item op1;
        item carry: bit = 'top.carry';
        cross opcode, carry;
    };
};
```

Analyzing Coverage

- **Help—**This helps you find the information you need in the Specman Online Documentation.
- **Coverage Extensibility—**This allows you to change coverage group and coverage item definitions.

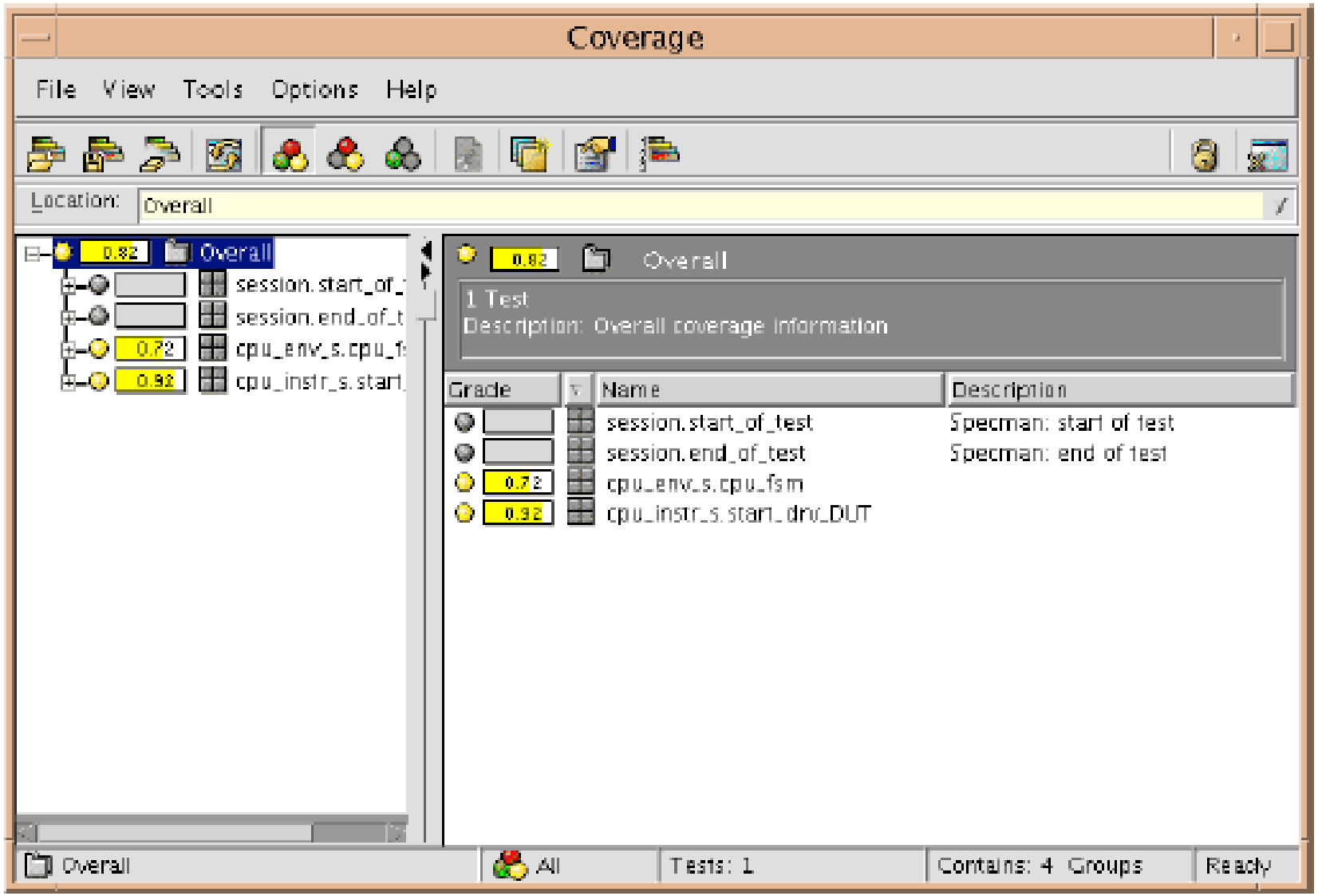
SimVision & (Specview) Command	How the Command is Used
Verification ›› Coverage (Tools ›› Coverage)	Displays coverage reports and creates cross-coverage reports.
Help ›› Specman Component of IES (Help ›› Specman Help)	Invokes the Specman Online Documentation browser (Cadence Help).

Analyzing Coverage

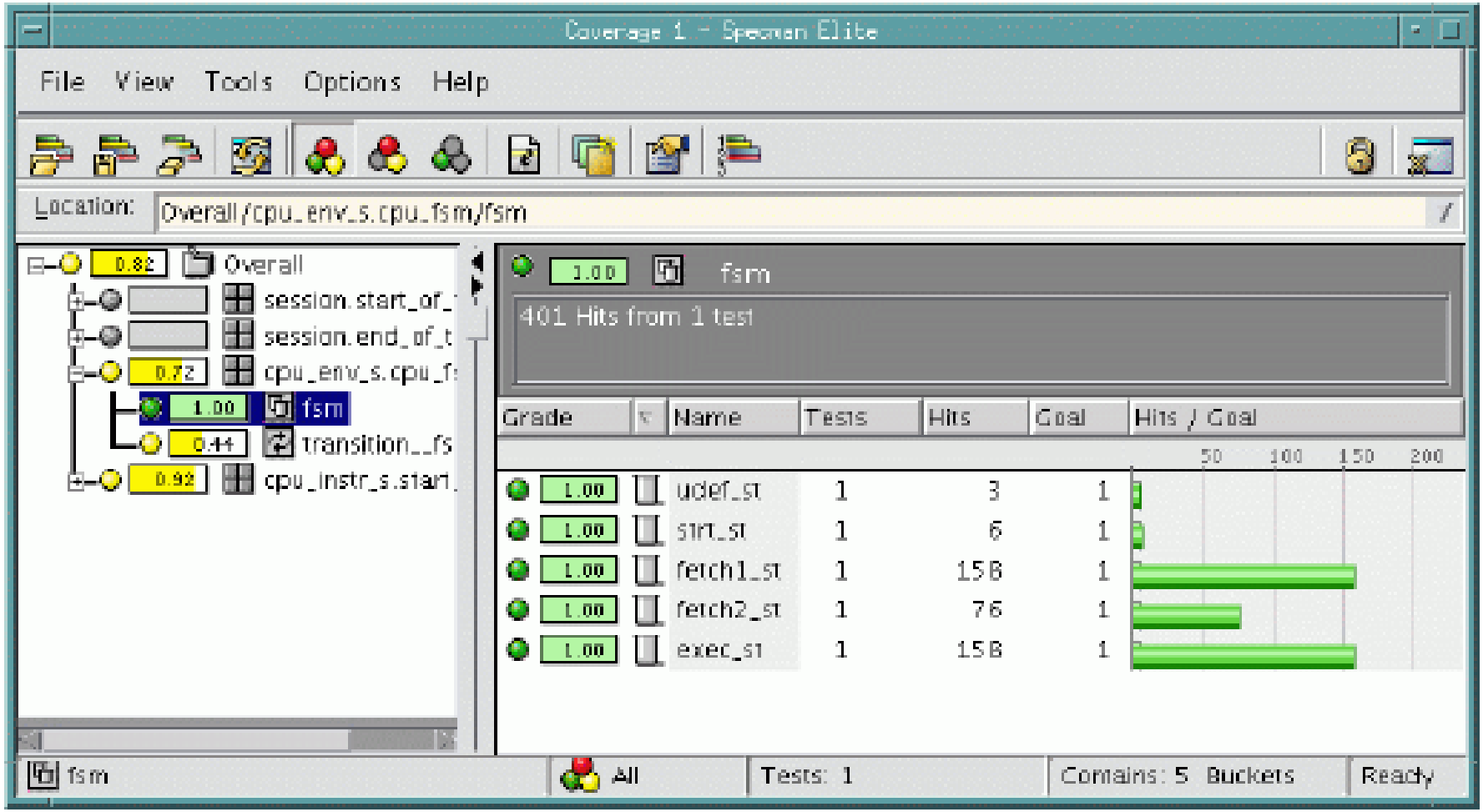
The steps required to analyze test coverage for the CPU design are:

1. Running tests with coverage groups defined.
2. Viewing state machine coverage.
3. Viewing instruction stream coverage.
4. Viewing corner case coverage.

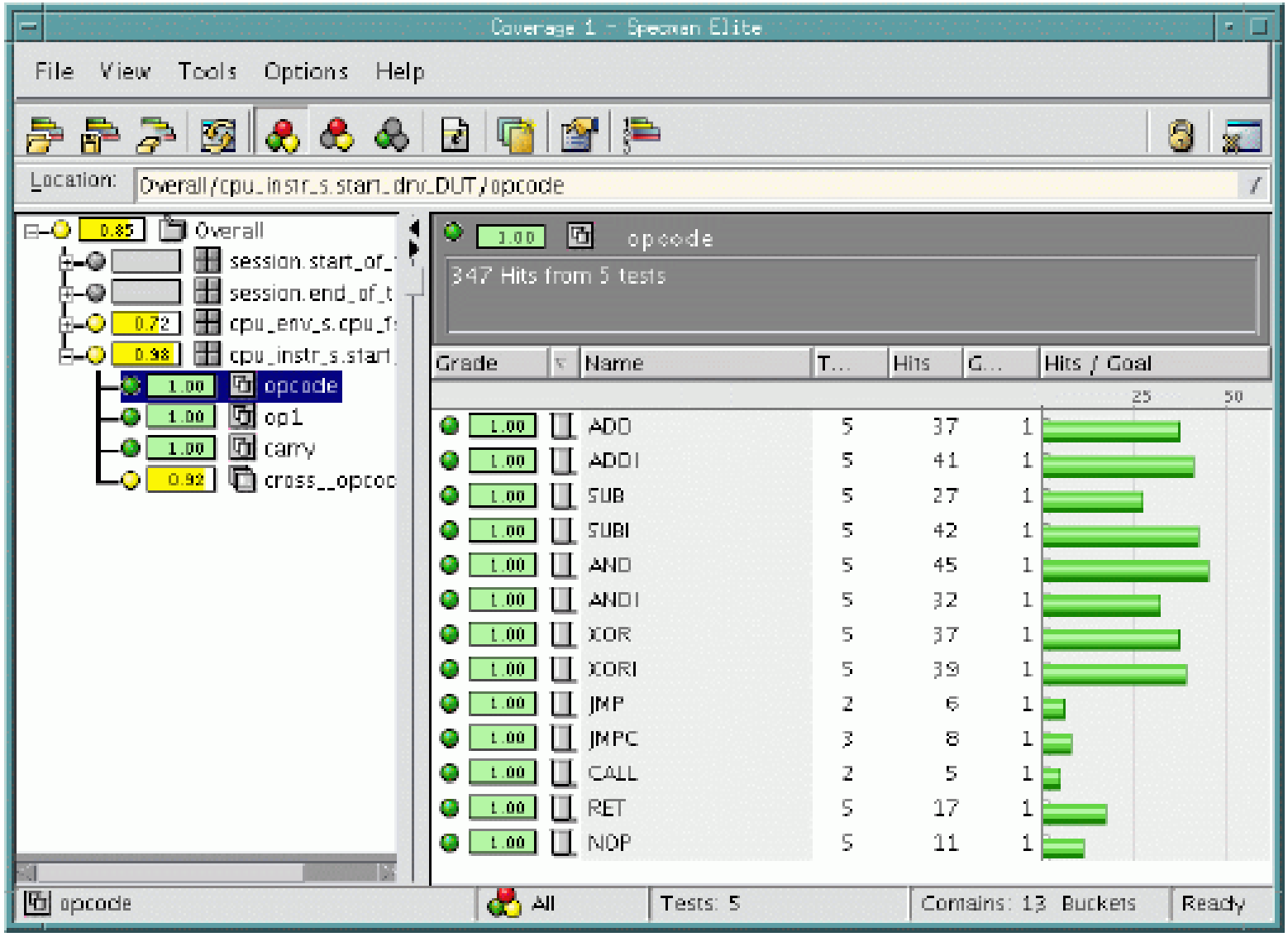
The Coverage windows



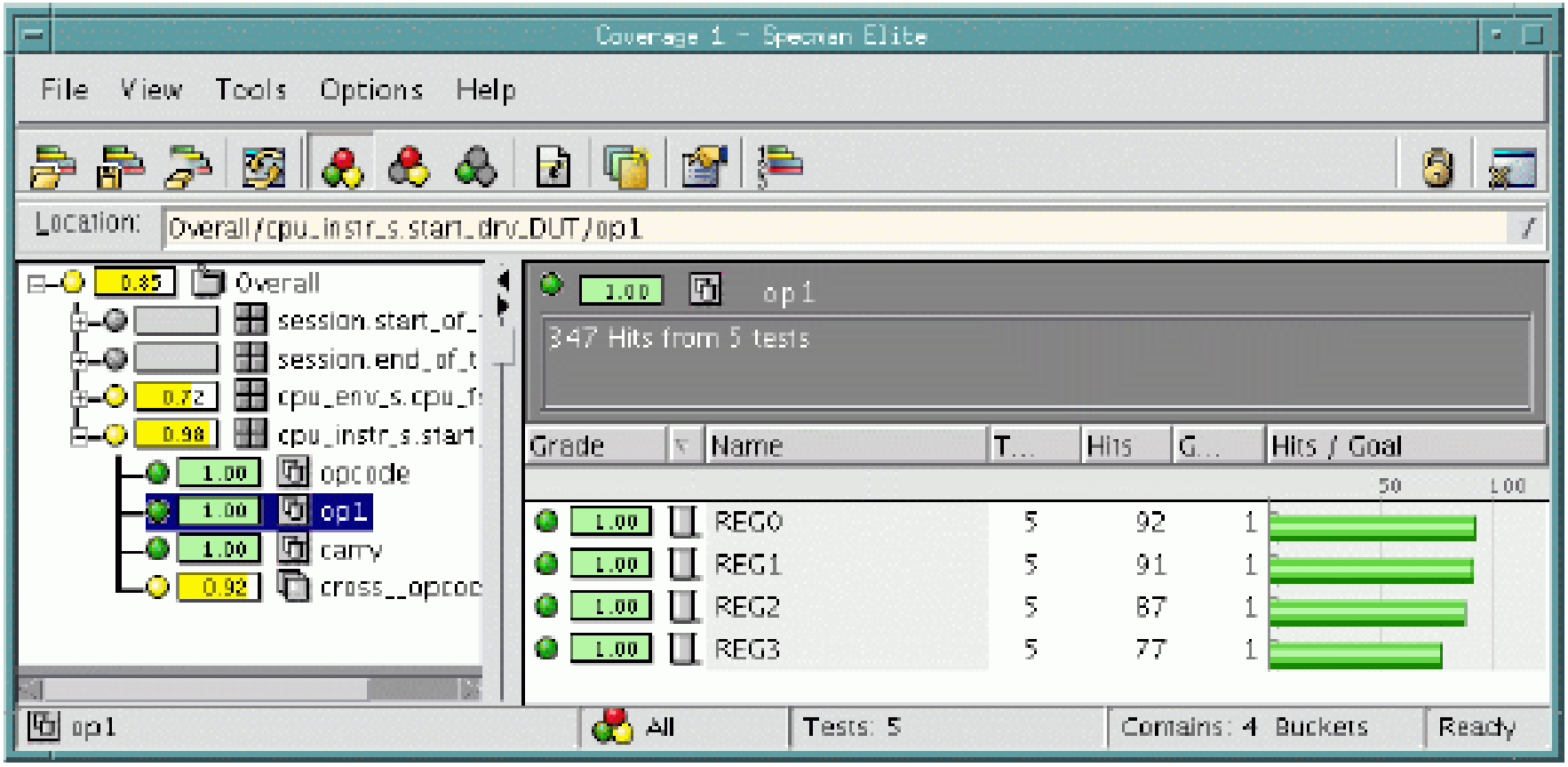
The Coverage windows



Viewing Instruction Stream Coverage



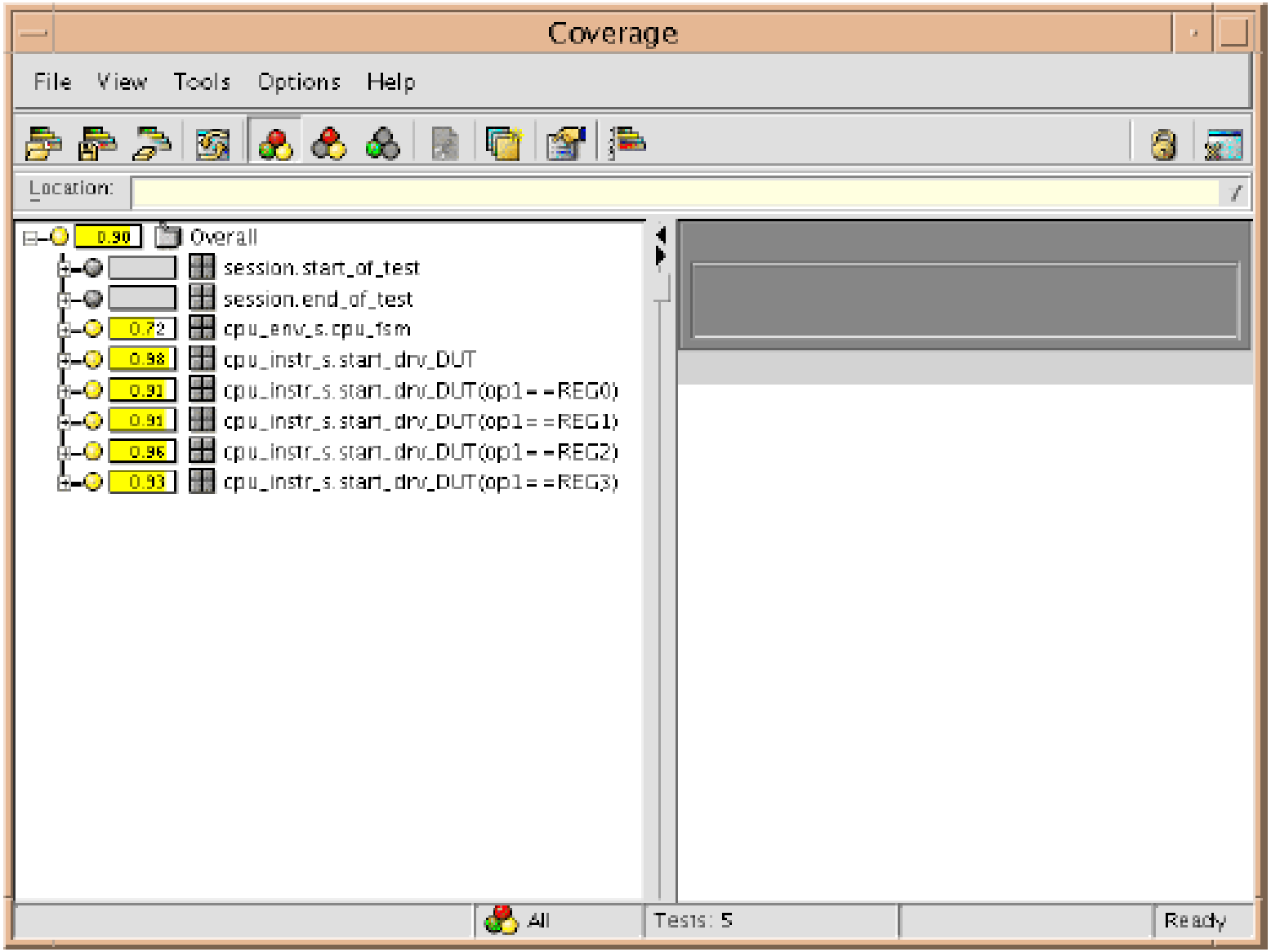
Viewing Instruction Stream Cov



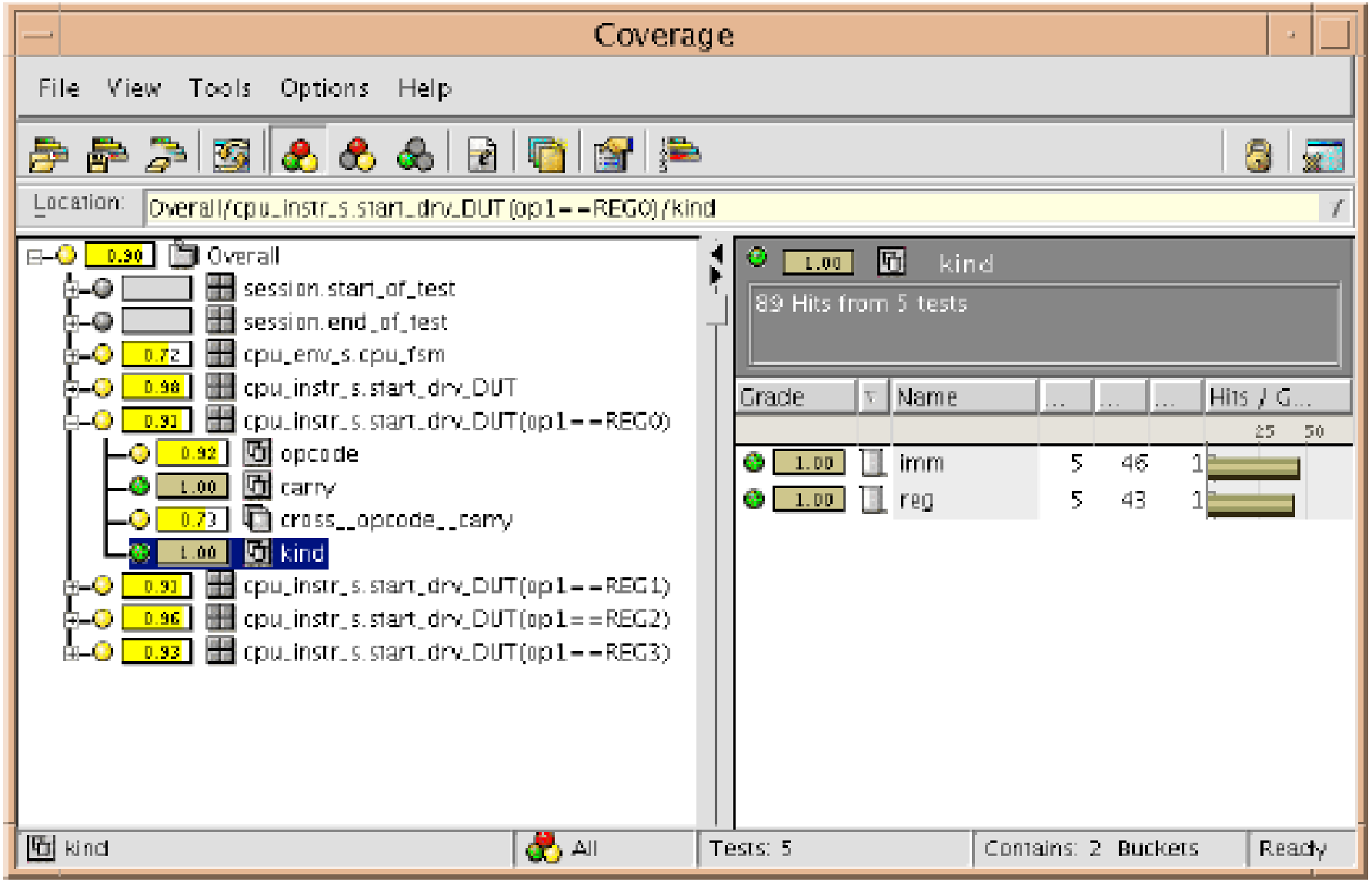
Extending Coverage

```
extend cpu_instr_s {

    // Extend the start_drv_DUT cover group with "is also"
    cover start_drv_DUT is also {

        // Add the kind field to the cover group as a new item
        item kind;

        // Extend the op1 item to make it a per_instance item
        item op1 using also per_instance;
    };
};
```

Viewing Coverage Per Instance



Viewing Coverage Per Instance

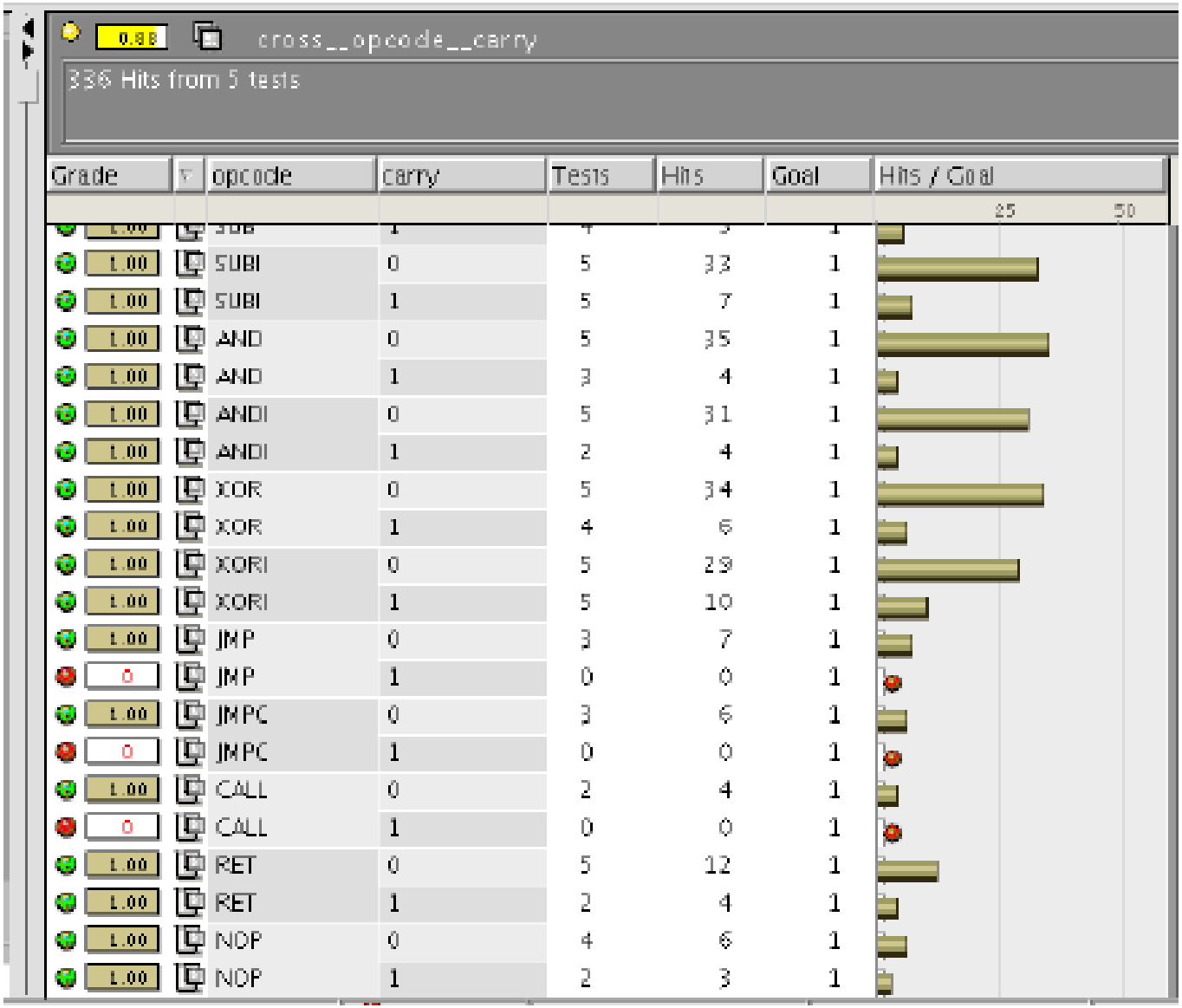


Viewing Corner Case Coverage

Our corner case coverage shows how many times the JMPC opcode was issued when the carry bit was high (1).

You can see, in the figure below, that carry was 0 each of the 6 times that opcode was JMPC. The combination of opcode JMPC and carry 1 never occurred in this set of 5 tests, which means that there is a coverage hole for that item and it has a grade of 0. The two other red items indicate other combinations of carry and opcode that never were hit in this set of tests. Any grade less than 1.00 is a hole. Grades less than 1.00 but more than 0 are shown in yellow in the coverage GUI.

Viewing Corner Case Coverage



Viewing Corner Case Coverage

The ability to cross test input with the DUT's internal state yields the valuable information that the tests created so far do not truly test the JMPC opcode.

You could raise the weight on JMPC and hope to achieve the goal.

However, many simulation cycles would be wasted to cover this corner case.

Writing a Corner Case Test

With Specman's **on-the-fly test generation**, you can direct the test to constantly monitor the state of signals in the DUT and to generate the right test data—at the right time—to reach a corner case scenario.

This feature spares you the time-consuming effort required to write deterministic tests to reach the same result.

Steps required to create the corner case test

The steps required to create the corner case test are:

- Increasing the probability of arithmetic operations.
- Linking JMPC generation to the DUT's carry signal.

Increasing the Probability of Arithmetic Operations

keeps high weight on arithmetic operations

```
extend cpu_instr_s {
    keep soft opcode == select {
        // high weights on arithmetic
        40 : [ADD, ADDI, SUB, SUBI];
        20 : [AND, ANDI, XOR, XORI];
        10 : [JMP, CALL, RET, NOP];

        // generation of JMPC controlled
        // by the carry signal value
    };
};
```

Linking JMPC Generation to the Carry Signal

- JMPC instruction into the DUT when the carry signal is asserted. A better approach is to monitor the carry signal and generate the JMPC instruction when the carry signal is known to be high.
- This methodology lets you reach the corner case from multiple paths, in other words, from different opcodes issued prior to the JMPC opcode. This test shows how the DUT behaves under various sequences of opcodes.

Linking JMPC Generation to the Carry Signal

```
extend cpu_instr_s {
    keep soft opcode == select {
        // high weights on arithmetic
        40 : [ADD, ADDI, SUB, SUBI];
        20 : [AND, ANDI, XOR, XORI];
        10 : [JMP, CALL, RET, NOP];

        // generation of JMPC controlled by
        // the carry signal value
        'top.carry' * 90 : JMPC;
    };
};
```

Creating Temporal and Data Checks

Specman **temporal constructs—**These powerful constructs let you easily capture the DUT interface specifications, verify the protocols of the interfaces, and efficiently debug them.

The temporal constructs minimize the size of complex self-checking modules and significantly reduce the time it takes to implement self-checking.

Creating Temporal and Data Checks

Specman **data checking—**Data checking methodology can be flexibly implemented in the Specman system:

- For data-mover applications like switches or routers, you can use powerful built-in constructs for rule-based checking.
- For processor-type applications, reference model methodology is commonly implemented.

The steps required to implement these checks

The steps required to implement these checks are:

1. Creating the temporal checks.
2. Creating the data checks.
3. Running the test with checks.

Construct	How the Construct is Used
expect	Checks that a temporal expression is true and, if not, reports an error.
check	Checks that a Boolean expression is true and, if not, reports an error.

Creating the Temporal Checks

The design specifications for the CPU require that after entering the *exec_st* state, the *fetch1* signal must be asserted in the following cycle. This is a temporal check because it specifies the correct behavior of DUT signals across multiple cycles.

Creating the Temporal Checks

```
// Temporal (Protocol) Checker
event enter_exec_st is
    (change('top.cpu.curr_FSM')and
    true('top.cpu.curr_FSM' == exec_st))
    @sys.cpuclk;

event fetch1_assert is
        (change('top.fetch1')and
        true('top.fetch1' == 1)) @sys.cpuclk;

//Interface Spec: After entering instruction
//execution state, fetch1 signal must be
//asserted in the following cycle.
expect @enter_exec_st => {@fetch1_assert}
    @sys.cpuclk else
    dut_error("PROTOCOL ERROR");
```

issues an error message if fetch1 does not rise exactly one cycle after entering execute state

Creating Data Checks

To determine whether the CPU instructions are executing properly, you need to monitor the program counter, which is updated by many of the control flow operations.

Reference models are not required for data checking. You could use a rule-based methodology.

However, reference models are part of a typical strategy for verifying CPU designs. The Specman system supports reference models written in Verilog, VHDL, C, or, as in this tutorial, ***e***. All you need to do is to create checks that compare the program counter in the DUT to their counterparts in the reference model.

Adding the Data Checks

issues an error if there is a mismatch in the program counters of the DUT and the reference model

```
// Data Checker
event exec_done is (fall('top.exec') and
    true('top.rst' == 0))@sys.cpuclk;

on_exec_done() is {
    // Compare PC - program counter
    check that sys.cpu_dut.pc ==
        sys.cpu_refmodel.pc else
        dut_error("DATA MISMATCH(pc)");
```

Synchronizing the Reference Model with the DUT

```
<'
import cpu_refmodel;

extend sys {
    event cpuclk is
       (fall('top.clk')@tick_end);

    cpu_env : cpu_env_s;
    cpu_dut : cpu_dut_s;
    cpu_refmodel : cpu_refmodel_s;
};
'>
```

creates an instance of the reference model

Resets the reference model

resets the reference model

```
reset_cpu() @sys.cpuclk is {
    'top.rst' = 0;
    wait [1] * cycle;
    'top.rst' = 1;
    wait [5] * cycle;
    sys.cpu_refmodel.reset();
    'top.rst' = 0;
};
```

```
    // execute instr in refmodel
    sys.cpu_refmodel.execute(instr,sys.cpu_dut);
};
```

Analyzing and Bypassing Bugs

- **The Specman debugger—**Provides powerful debugging capability with visibility into the HDL design.

- **The Specman bypass feature—**Lets you temporarily prevent the Specman system from generating test data that reveals a bug in the design. With this feature you can continue testing while the bug is being fixed.

Generation Debugger Menu

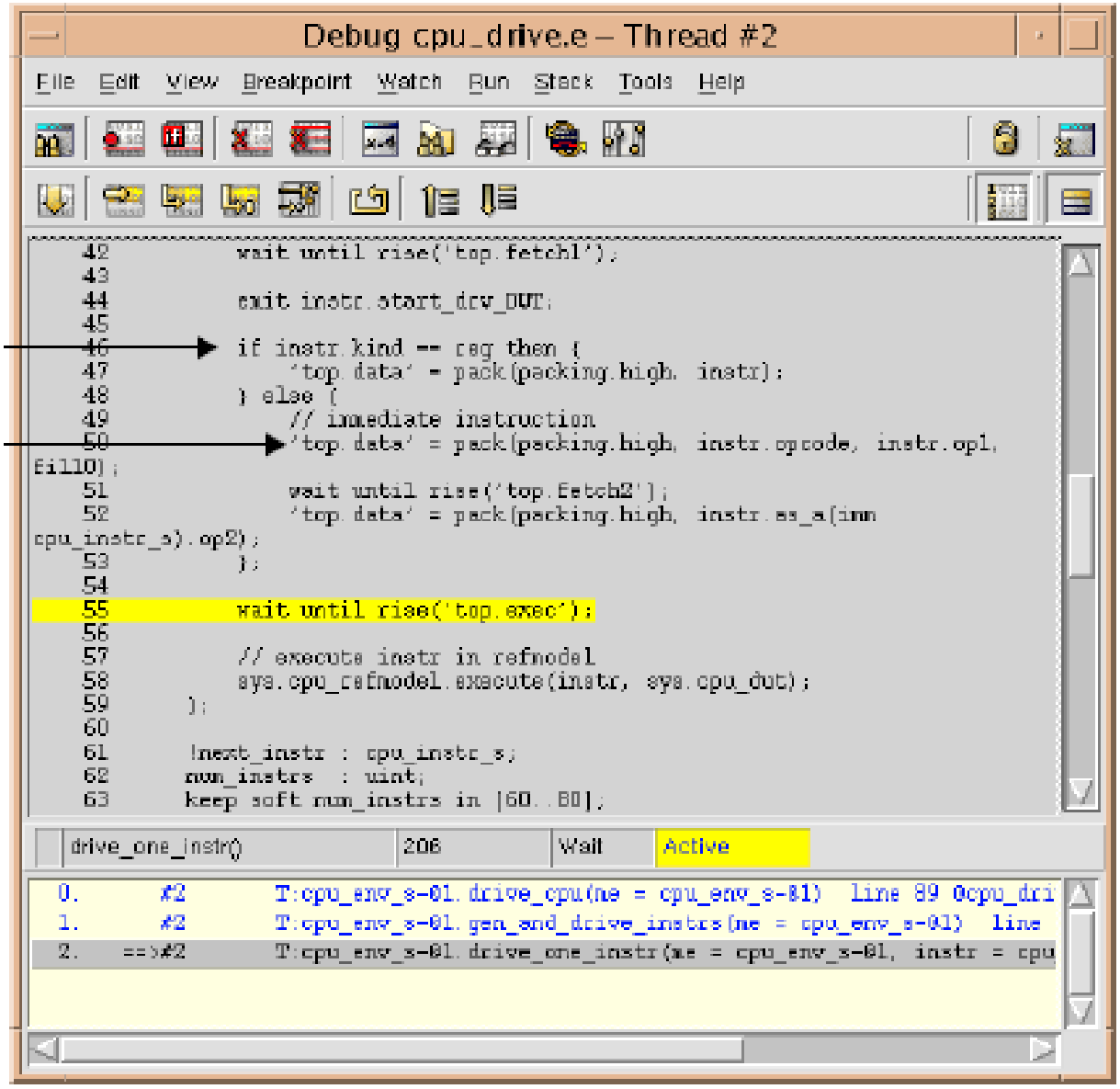
Generation Debugger Command	How the Command is Used
View ›› Print	Displays the current value of an *e* variable.
Breakpoint ›› Set Breakpoint ›› Break	Sets a breakpoint on the currently highlighted line of *e* code.
Run ›› Step Any	Advances simulation to the next line of *e* code executed in any thread.
Run ›› Step	Advances simulation to the next line of *e* code executed in the current thread.

The steps for debugging the temporal error

The steps for debugging the temporal error are:

1. Displaying DUT values.
2. Setting breakpoints.
3. Stepping the simulation.
4. Bypassing bugs.

Displaying DUT Values



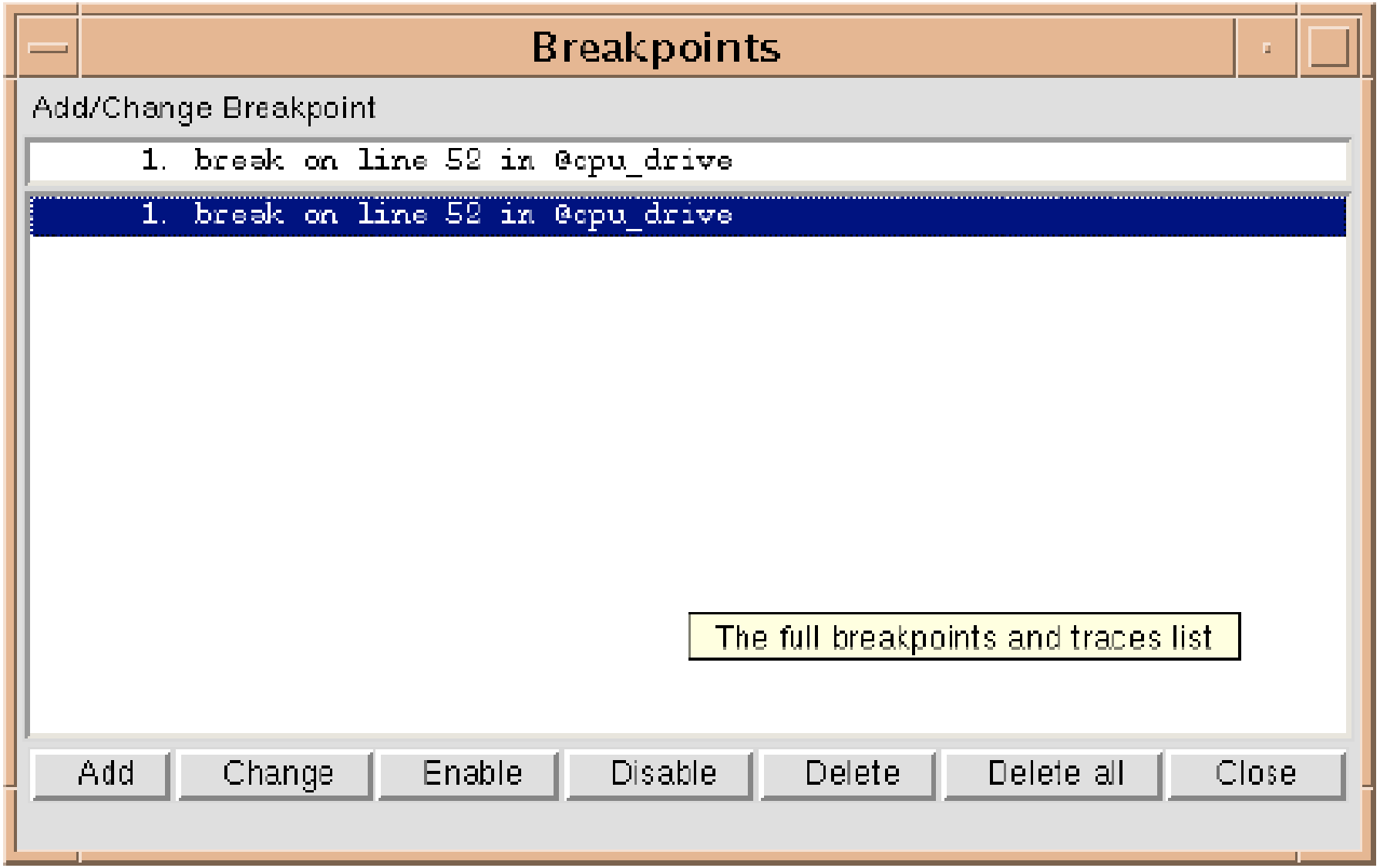
Setting Breakpoints

You have determined that the bug appears on an immediate instruction when the opcode is JMPC.

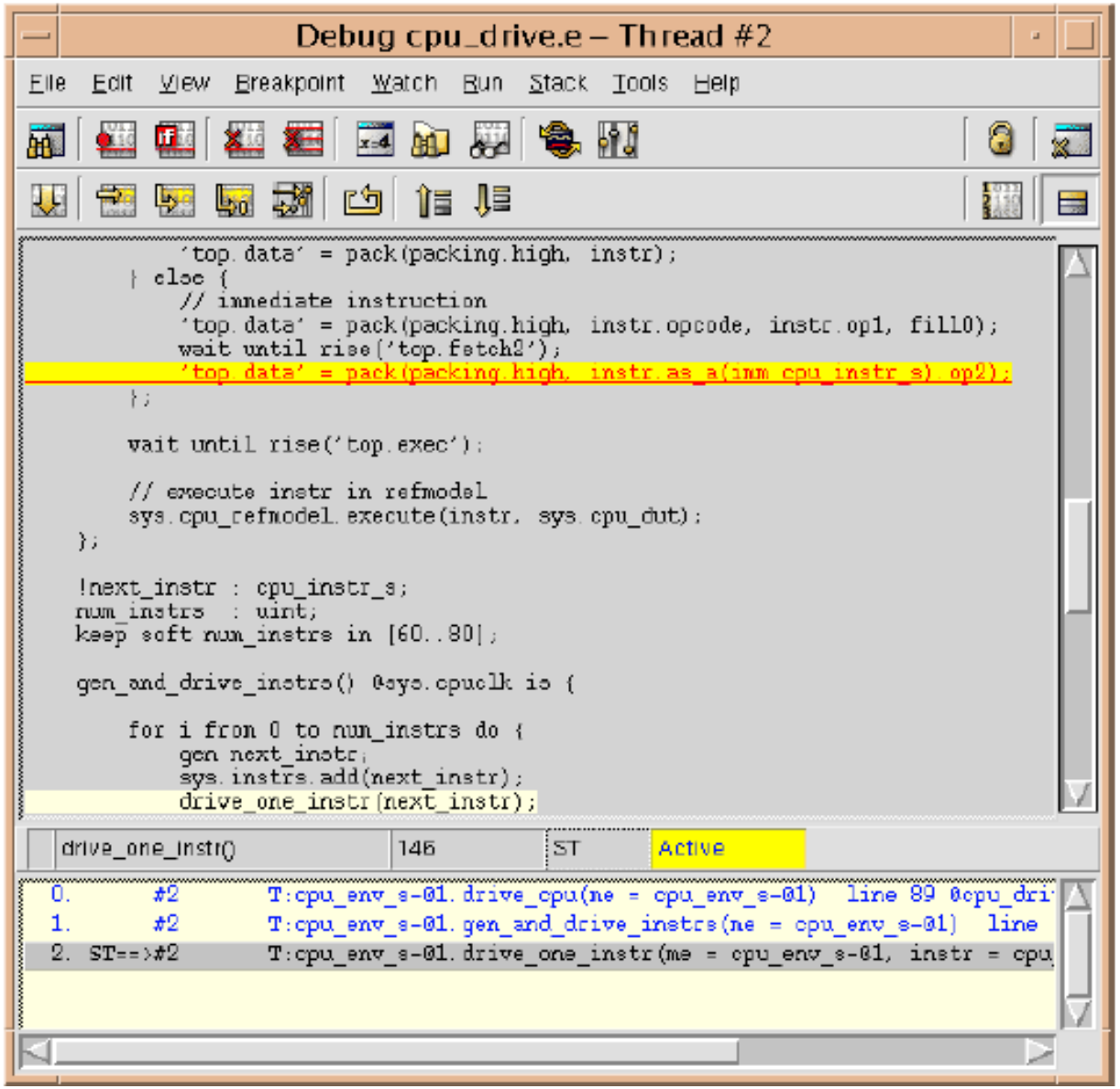
It may be possible to narrow down even further the conditions under which the bug occurs.

You can set a breakpoint on the statement that drives the immediate instruction data into the DUT to see what the operands of the instruction are.

Setting Breakpoints



Stepping the Simulation



Bypassing the Bug

A common problem in traditional test generation methodology is that when there is a bug in the design, verification cannot continue until the bug is fixed.

There is no way to prevent the generator from

creating tests that hit the bug.

The Specman system's extensibility feature, however, lets you temporarily prevent generation of the conditions that cause the bug to be revealed.

This particular bug seems to surface when the JMPC operation is performed using a memory location greater than 10. To continue testing other scenarios, you simply extend the test constraints to

prevent the Specman system from generating this combination.

To bypass the JMPC bug

```
<'
extend imm cpu_instr_s {

    keep (opcode == JMPC) => op2 < 10 ;

};
'>
```